GROUP 2 VLSI PROJECT REPORT

Victor Basvi Progress Munorairwa Ben Keppers

DECEMBER 11, 2025

EE5900

Summary

This project presents the design, implementation, and verification of a 4-bit MIPS-lite RISC processor using the Skywater 130nm PDK in Cadence Virtuoso. The processor executes a reduced instruction set including READ, WRITE, ADD, SUB, AND, OR. SHIFT LEFT and SHIFT RIGHT operations, targeting hierarchical, full-custom CMOS implementation. Key contributions from the team include the design of the control logic state machine, the register file with three independent 4-bit registers, the Command execution block/arithmetic logic unit (ALU) supporting all the required operation, and necessary multiplexing and datapath components. A three-level hierarchical methodology, from transistor level standard cells to functional blocks to full system integration, was followed, with each stage verified using Pegasus DRC, ERC, and LVS. The final integrated layout demonstrates successful physical design rule compliance, logical correctness, and floorplan-aware integration, fulfilling graduate-level requirements for a complete VLSI design cycle from architecture to GDSII ready layout.

Introduction

The design of modern microprocessors represents one of the most complex challenges in Electrical engineering, integrating architecture, logic design, and physical implementation into a single silicon chip. This team-based project applies to the principles of Very Large-Scale integration (VLSI) to design a simplified 4-bit MIPS-lite RISC processor, inspired by the MIPS architecture outlined in Waste & Harris. The MIPS (Microprocessor without Interlocked Pipeline Stages) architecture is a seminal RISC design characterized by a streamlined instruction set, uniform length, and a load-store architecture, making it ideal for exploring digital system design at the transistor level.

The primary objective of this project is to exercise the complete VLSI design flow, from architectural specification and logic design through schematic capture, physical layout, and final verification, using industry standard EDA tools. Specifically, the project uses Cadence Virtuoso for schematic and layout design and Cadence Pegasus for physical verification (DRC, LVS, ERC), all within the realistic design constraints of the open source Skywater 130nm process design kit (PDK). This PDK provides a real-world technology base, introducing practical considerations of design rules, parasitic effects, and layout optimization that are absent in purely behavioral simulations.

The processor implements a graduate level instruction set of eight commands operating on a 4-bit data path with three registers. A hierarchical design methodology is emphasized, requiring three levels of design: foundational standard cells (e.g. D flip-flops, logic gates), functional blocks (registers, Command execution block/ALU, multiplexers, control logic), and the top-level system integration. This structure not only manages complexity but also reinforces the importance of reuse and incremental verification in large-scale digital design.

Beyond functional correctness, the project emphasizes physical design integrity. Floor planning, power distribution, signal routing, and design rule compliance are treated as first class design constraints, mirroring industry practice. The outcome is a fully laid-out processor that is both logically verified and physically manufacturable, demonstrating the integration of architectural insight, circuit design skill, and physical implementation demand expected of graduate-level VLSI students. The report documents the team’s collaborative journey, detailing the design motivations, topological chives, implementation strategies, and verification results of the final layout.

Design

The datapath topology choice for this design was heavily based on the command set provided as a design requirement. A common architecture for MIPS-based processors includes a register bank, an ALU, and a memory bank that are set up so the output of each is fed to the input of the next [1-3]. The command set, shown in Table 1, consists only of commands that perform a single action based on the value in one or both registers, and there is no support for storing an ALU result directly to memory within the same command. Because of this the typical structure of an ALU feeding into a memory bank was not followed.

Table 1: Command Set

Command	Code	Description
ADD	0000	Reg A + Reg B = Reg C
SUB	0001	Reg A - Reg B = Reg C
AND	0010	Reg A & Reg B = Reg C
OR	0011	Reg A | Reg B = Reg C
READ	0100	Read from Memory
SR	0110	Reg A >> Reg B = Reg C
WRITE	1000	Write to Memory
SL	1100	Reg A << Reg B = Reg C

The initial design plan for this project, Figure 1, consists of two input registers, an ALU and Memory in parallel, and an output register. A multiplexer was used to select between ALU and memory outputs, and a demultiplexer was used to select which block the input registers connect to. This demultiplexing step was not found in other designs that were investigated [1-3]. The design rationale for this step was that the demultiplexer reduces the electrical effort at the input registers at the cost of increasing path length.

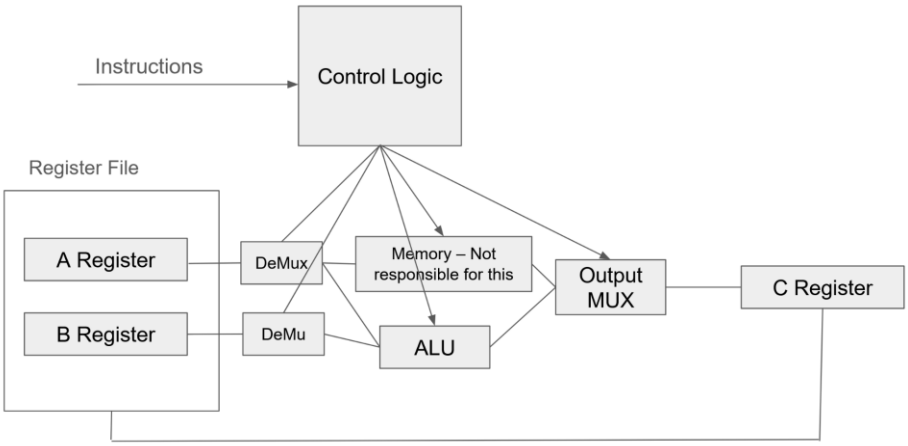

Figure 1 Initial MIPS processor block diagram

Ultimately, the demultiplexing stage was abandoned to simplify layout for the design. The distinction between memory and the ALU was also eliminated to allow it to be more closely integrated with the ALU block multiplexers, resulting in the design shown in Figure 2. This combined memory/ALU is referred to as the "Command Execution Block".

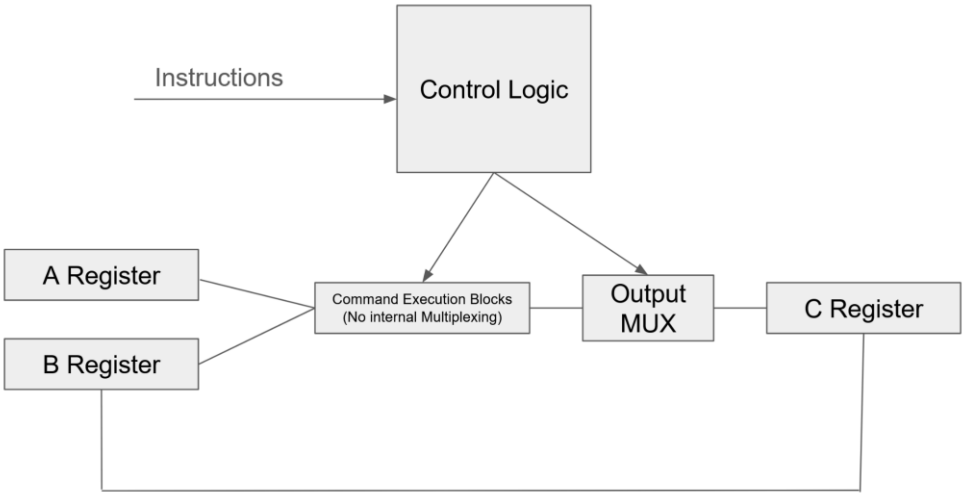

Figure 2 Final MIPS processor block diagram

The command execution block encompasses the ALU and memory in this design. For all intents and purposes, it is an ALU, but it also contains the memory. It consists of five functional blocks including ADD/SUB, AND, OR, SL/SR, and memory. For the purposes of this project a single 4-bit register is used to emulate memory, enabling the design to support memory with only the addition of memory and an addressing system.

There are two common ALU structures including the tree structure and the chain structure, Figure 3. The tree structure consists of all ALU blocks feeding their outputs to the same multiplexer [4]. In the chain structure, only 2:1 multiplexer is used, and they are chained into each other [4].

Typically, the tree structure is faster at the cost of using more area for layout [4]. The chain structure was chosen for this design for several reasons. First, the chain structure requires only 2:1 multiplexers, eliminating the need to complete the layout of a large 8:1 multiplexer. The chain structure also allows for a more flexible number of blocks, allowing for only 5 blocks without having 3 unused inputs. Finally, the multiplexing is dispersed between multiple smaller multiplexers in the chain structure. That means multiplexers can be placed closer to the blocks they have as inputs, reducing the routing distance for unwieldy 4-bit data lines.

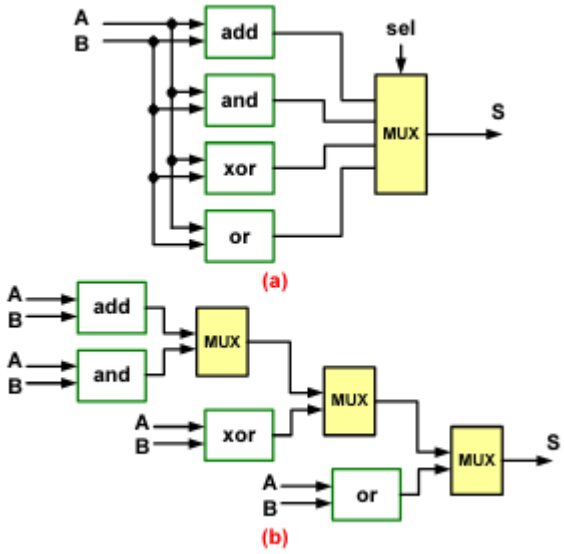

Figure 3 (a) Tree ALU structure (b) Chain ALU Structure

One downside of using a chain multiplexing architecture is the varied delay path that stems from the signal propagating through a variable number of multiplexers depending on which ALU block is being used. Fortunately, this design consists of blocks with a varying level of complexity, and the delay can be somewhat equalized by placing more complex blocks on a lower delay path and by placing fewer complex blocks on a higher delay path. Propagation delay simulations were performed for each block to determine the optimal order for them. In the end, the tree multiplexing structure was selected for this design due to the simpler operation and schematic design.

Experiment

Design of each functional sub cell was completed based on a variety of reference designs. First, a collection of basic logic gates was designed so they could be used for higher level designs. The main functional blocks were then designed with the focus of creating a simple yet functional design.

Bitwise Logic

4-Bit Bitwise OR Gate

The OR function (0011) is implemented as a dedicated combinational block within the ALU. It was constructed hierarchically from pre-designed standard cells contributed by team members:

- Core logic: A 2-input NOR gate followed by an inverter creates a basic 2 input OR gate
- 4-bit expansion: This single bit OR cell was replicated four times to create the full 4-bit bitwise OR gate, with each bit operating independently on corresponding bits from Registers A and B.

The structured gate level approach was chosen over custom transistor level OR design because verified standard cells (NOT, NOR) accelerated design and ensured seamless integration since all cells were designed to the Skywater PDK design rules.

The OR gate follows standard cell height ensuring alignment in the final floorplan. Transistor sizing was balanced to match the drive strength of the 4-bit data bus.

4-Bit Bitwise AND Gate

The AND function (0010) was implemented using four 2-input AND gates that were designed for use in another portion of the design. Layout was very simple for this block, as layout had already been completed for the AND gate. All that remained was to place the four gates and route wires between them.

Addition and Subtraction

A 1-bit full adder was designed based on the mirror adder topology presented in the course textbook Figure 4 [5]. Four mirror adders were then cascaded with the Cout bit of one feeding into the C input of the next. To enable subtraction via two's complement, two-input XOR gates were added on the B input with the second input connected to an active high subtraction signal. The subtraction signal was also fed to the carry input of the 4-bit full adder (Figure 5).

When working through layout for the addition block it was determined that incorporating the XOR gates into the 1-bit full adder layout would greatly reduce layout effort, so the hierarchy was adjusted accordingly.

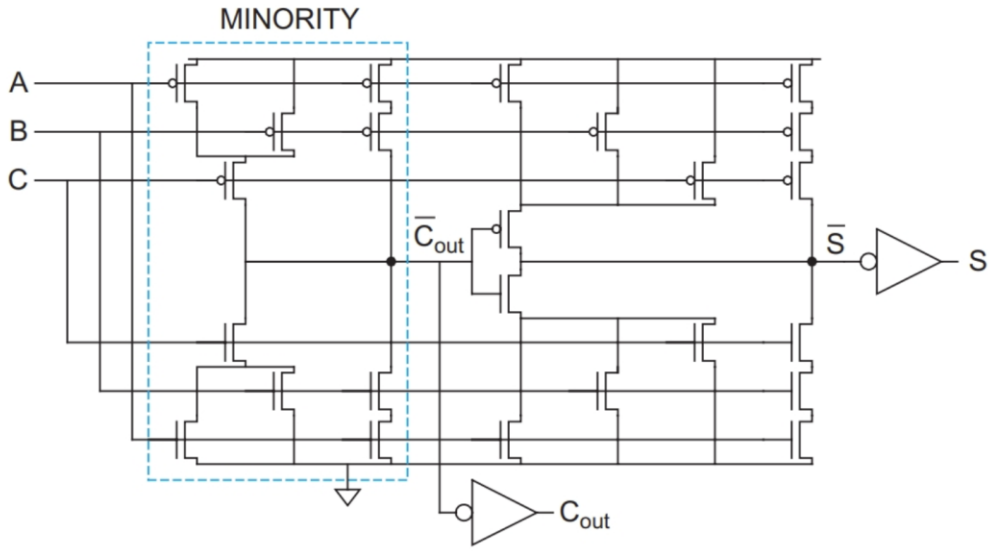

Figure 4 Mirror adder topology for a 1-bit full adder

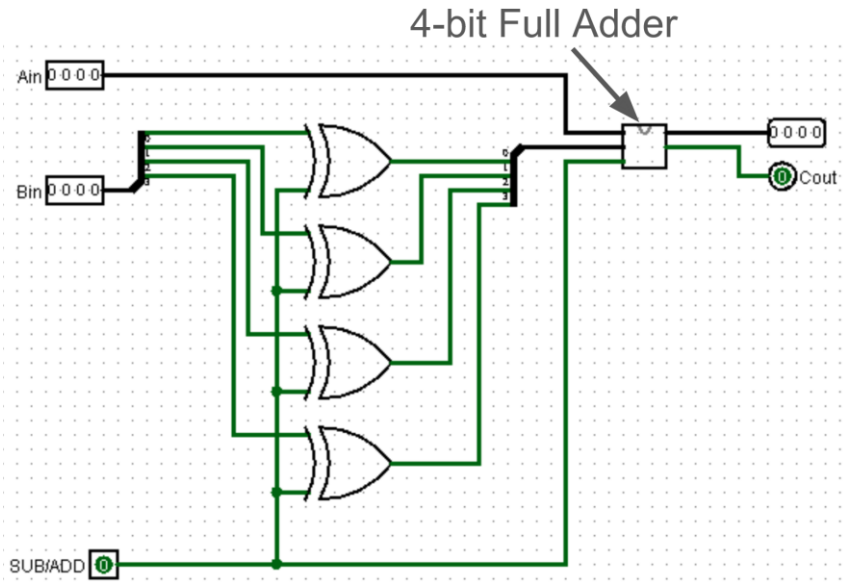

Figure 5 Addition and subtraction block design

4:1 MUX

A multiplexer is a combinational circuit that has many data inputs and a single output, depending on control or select inputs. For N input lines, log2(N) selection lines are required, or equivalently, for 2n input lines, n selection lines are needed meaning for a 4:1 MUX we need 2 select lines. It works by selecting one of four input signals and forwards it to a single output line based on the binary combination of two selection inputs S1 and S0.

In our project, the 4×1 MUX serves as the core building block of the 4 bit universal barrel shifter, where we used 4x1 MUX units to operate in parallel to handle the four data bits simultaneously, one for each output bit y3 to y0. The selection lines S0 and S1 control all MUXs simultaneously, ensuring synchronized shifting or rotation of the input word. Because of the MUX's single-cycle selection capability, the overall barrel shifter achieves parallel data shifting without introducing clock delays, resulting in significantly lower power consumption.

Left and Right Shifting (Barrel Shifter)

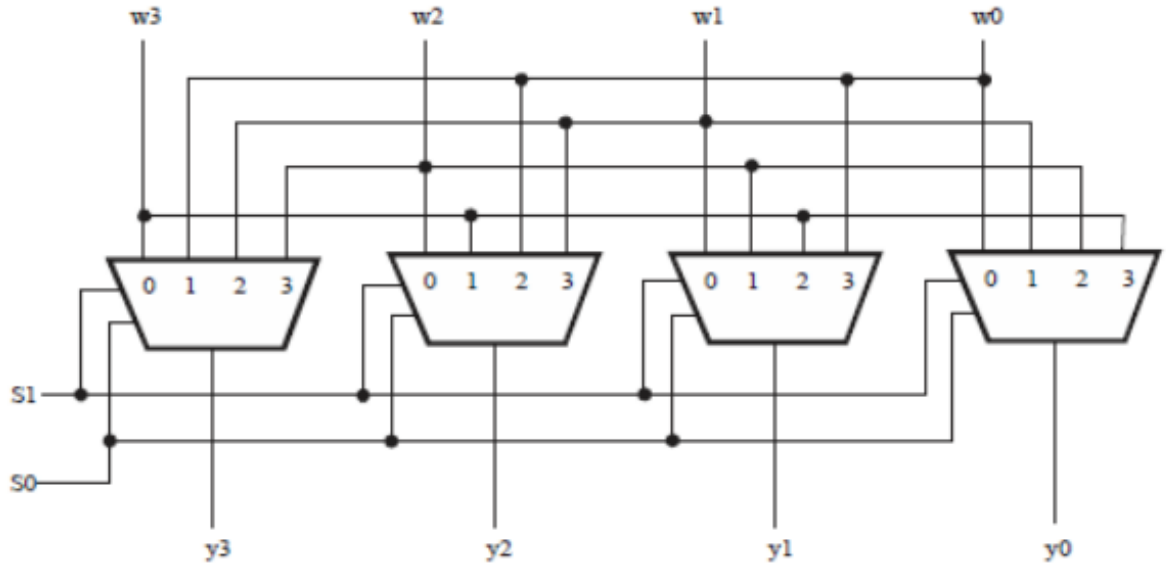

Figure 6 Barrell Shifter Design (source: https://ijrti.org/papers/IJRTI2306112.pdf)

We implemented our barrel shifter using four 4:1 MUX and each MUC is responsible for producing one of the four output bits (y_3, y_2, y_1, y_0) corresponding to the four input bits (w_3, w_2, w_1, w_0). Two control lines, S_1 and S_0, they determine the amount of shift or rotation to be performed.

The circuit can perform shifts of zero, one, two, or three bits in either direction. This operation is achieved instantaneously, within a single clock pulse, by using a cascade of MUXs where each output line selects one of the input lines according to the control word. Figure 6 presents this

configuration as a universal shifter, capable of both left and right rotations and arithmetic or logical shifts through appropriate control signal assignment (Praveen, Manasa, Chandana, & Jyothsna, 2023).

We chose this design specifically because unlike conventional serial shifters which require multiple clock cycles the barrel shifter performs all shifting and rotation operations within a single clock cycle, significantly improving data throughput and reducing latency

Register File

The register file forms the data storage core of the processor, containing three independent 4-bit registers (A, B and C) as required by the MIPS-lite specification. The design follows a strict hierarchical methodology to ensure ease of verification and reusability.

The fundamental building block is a D Flip-flop with synchronous enable, implemented in CMOS. This topology was chosen over latches for its robustness in synchronous systems as it prevents transparency issues and guarantees data capture on the rising clock edge thereby simplifying control timing. The enable logic is integrated using minimal additional transistors, ensuring compact layout.

Using Cadence Virtuoso, a 1-bit register slice was created by instantiating a D flipflop cell. This slice was then replicated four times to form a 4-bit Register. Finally, three instances of this 4-bit Register were placed to create the complete register file (A, B, C) in the final schematic design, shown in Figure 22.

An important architectural decision was the elimination of the 1-to-8 demultiplexer. Instead, the control unit generates three independent write enable signals. This simplification was possible because our register file provides four independent read ports (A and B to the ALU, C to the shifter) , eliminating the need for complex output multiplexing. This reduced control logic complexity at the expense of additional control wires.

The layout was crafted using the Skywater 130nm PDK with a standard cell row-based methodology. DFF cells were designed to a fixed height with aligned power (VDD) and ground (VSS) rails, allowing clean abutment when tiled. For the 4bit register layout design, L1M1 vias and Li layer were used where inputs and outputs were crossing the VDD, Clock and ground rails as these rails run vertically on M1 as shown in Figure 23.

Control Logic

The control logic schematic was designed in Logisim using the logic generation function before transferring the schematic to Cadence. The control logic has one 4-bit input that accepts the command being sent to the processor. It then needs to output one multiplexer select bit for each of

the four 2:1 multiplexers, a shift direction bit, an add/subtract selection bit, and a memory write bit. The requirements for these signals were entered into a table using the values 1, 0, and x (don't care). Logisim was used to generate a logic circuit using only 2-input NAND gates. This produced a functional circuit, but many NAND gates were being used as inverters, and there were several locations where the gate could be reduced by replacing NAND and NOT gates with a NOR gate. The resulting circuit was tested to verify it still gave the correct outputs, and it was transferred to Cadence.

The control logic circuit wound up being made of three blocks of logic gates along with a collection of NOT gates. Layout was first done to minimize the size of the three logic gate blocks, those blocks were placed close together, and NOT gates were used to fill in gaps. Gates were mirrored and rotated such that the nmos and pmos ends were adjacent. Four vertical metal 2 wires were run vertically on the left side of the layout to serve as an input bus, and all gates that took command bits as inputs were wired directly to that bus. Finally, to keep layout as simple as possible, metal 1 was used for all horizontal routing and metal 2 was used for all vertical routing.

2:1 MUX

A critical component in the data path, this multiplexer selects between the ALU output and memory data for register write-back operations. The topology is chosen for its low propagation delay and minimal transistor count. It uses a NOT gate connected at the NOR output to make an OR gate, then two AND2 gates serve as the inputs to the NOR gate and a NOT gate as the selector. In the final system schematic, this MUX resolves a key data routing requirement: during READ operations, data from the memory interface is selected; during arithmetic/logic operations, the ALU result is selected. This clean separation maintains the RISC principle of separating memory and computational pathways.

The layout was optimized for compactness and alignment with other blocks. Inputs, outputs, and the select signal were placed to facilitate straight routing in the top-level floorplan.

Results and Discussion

Layout Challenges

During the physical layout we faced a few challenges. One of the most persistent issues has been the misalignment of pins and interface terminals between blocks created by different team members. Even small deviations in pin spacing, metal layer choice, or orientation complicated the top level routing and led to connectivity errors during integration. This required us to repeatedly revise symbol conventions and manually adjust pin placements to ensure consistency across all blocks.

Debugging DRC and LVS errors has also proven difficult, primarily because resources for Sky130 layout troubleshooting are limited, and many error codes require deep interpretation of design rules rather than straightforward fixes. This results in a time consuming iterative process especially when we were doing the layout of the full project. Identifying layout violations, understanding the underlying rule conflict, and correcting the placement without introducing new errors have been really difficult. We also faced the challenge of optimizing the area while trying not to violate any design rules. Because achieving a compact layout is essential for a clean floorplan, yet aggressive compaction can create spacing violations, enclosure errors which we faced quite a lot, or unintended diffusion merges. Other layout related complications included managing well spacing to avoid latch up risks, ensuring proper power rail continuity across cells, and maintaining balanced routing to minimize parasitics. These challenges show the need for careful planning, consistent design across all group members and continuous communication to achieve a functional, clean layout.

Design Rule Checking (DRC) and Layout Versus Schematic (LVS)

When running the DRC for the full schematic, some changes were required to make the design pass. The final layout passes all DRC checks except “licon.12”. This rule violation highlights regions of merged transistors, giving the error “Max SD width without licon <= 5.700 um” (Figure 7). This rule violation appears to be caused by an error in the rule checking, as the largest transistors that are highlighted are on the order of one micron at the largest dimension. That is significantly smaller than the dimension given in the violation description.

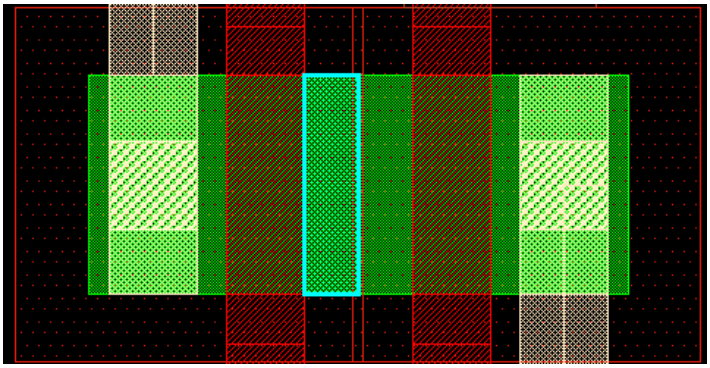

Figure 7 Example of the area highlighted for licon.12 rule violations

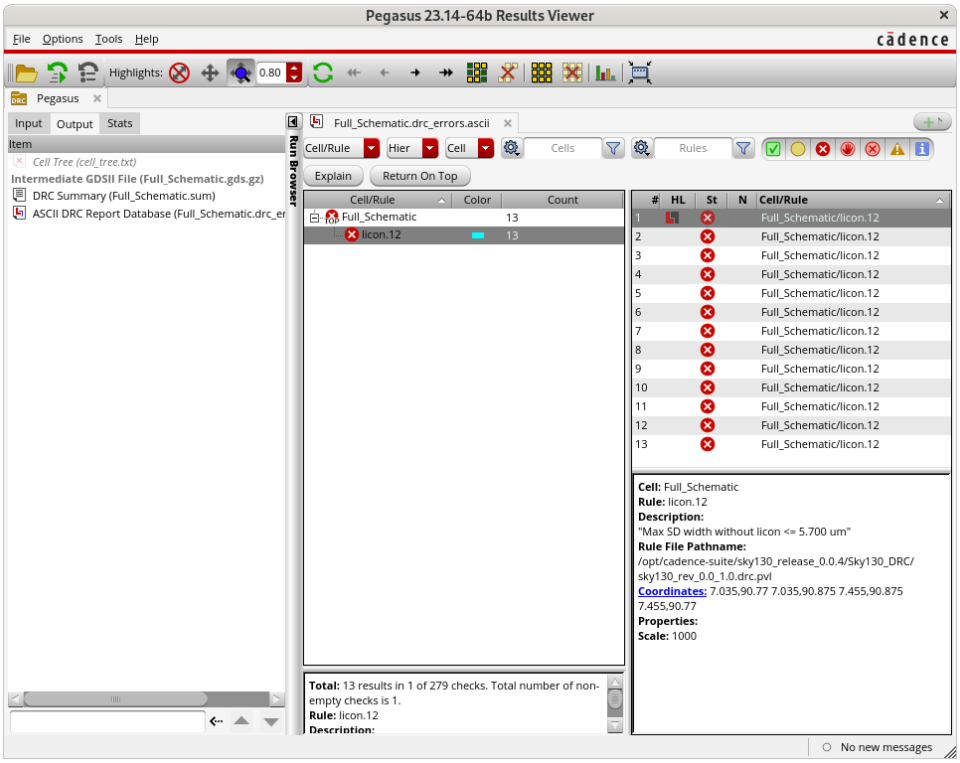

Figure 8 DRC Results

The LVS revealed some open and short connections throughout the design, but these were solved without too much issue. Another issue in the LVS was that pins must be placed on the Metal 1 Pin layer for Pegasus LVS to find the pins. After solving those issues the LVS was passed successfully as shown in Figure 9.

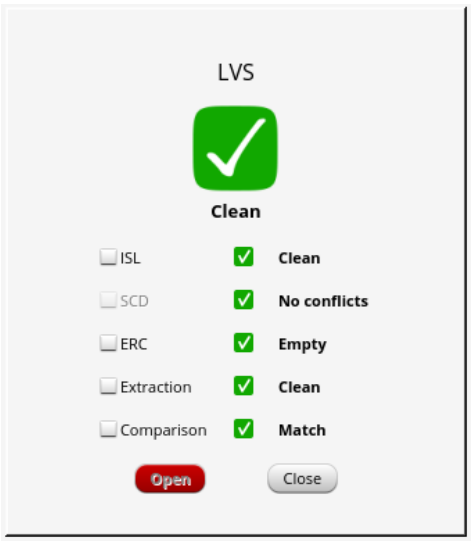

Figure 9 LVS Results with no errors

Simulated Block Delays

In order to determine the best order for blocks in the multiplexing chain, simulations were run for each block to measure the delay from the input rising edge to the output rising edge. This was then taken to be the propagation delay (Table 2). The optimal gate order would be to have the block with the highest propagation delay at the lowest delay position in the chain and the lowest delay blocks at the highest delay position in the chain.

Table 2:

Gate	Delay
AND	41.36 ps
OR	34.2 ps
Register CLK to Q	108.5 ps
ADD	267.6 ps
Shift	219.6 ps

Functionality Simulations

Design functionality was verified with simulation in which the A and B register inputs were set to constant values, and the command input was cycled through all possible inputs. The A and B registers were set to the values 0111 and 1010 respectively, as required from the project specification document. The outputs were then observed for each valid command input, as shown in Figure 10. The simulation was run through

one entire cycle of commands before taking the output, as the memory read command is tested before the write command. Another design note is that the output register is fed an inverted clock signal to allow the A and B registers to update at the start of each clock cycle. Because of that, the output is only valid after the falling edge of the clock. Some troubleshooting efforts were required after the first simulation was run; however, the issues were quickly resolved to yield the correct outputs for all command inputs.

Table 3: Expected simulation results

Command	Command Byte	Output
ADD	0000	0001
SUB	0001	1101
AND	0010	0010
OR	0011	1111
READ	0100	0111
WRITE	1000	-
Shift Right	0110	1011
Shift Left	1100	1110

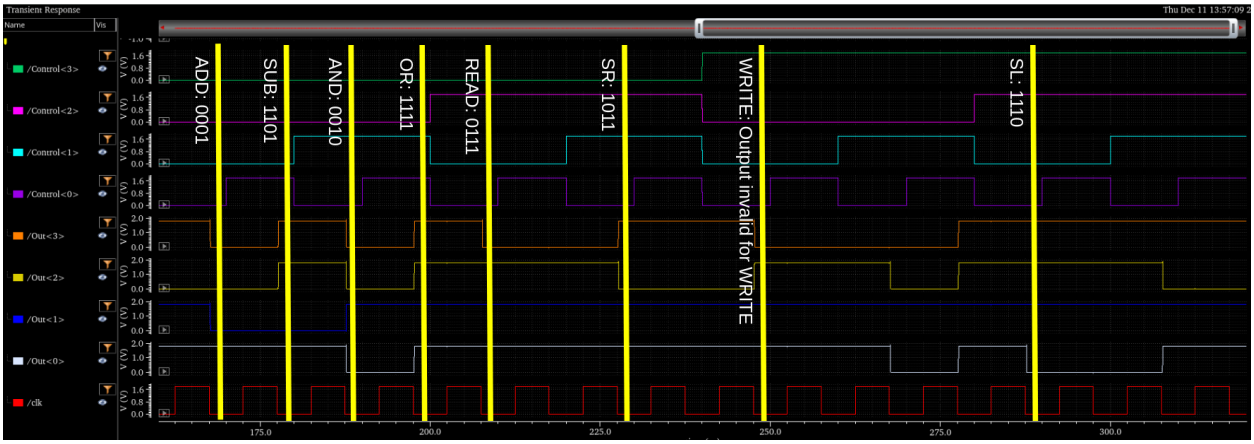

Figure 10 Simulation results from full schematic simulation

Conclusions

This project successfully achieved the design, implementation, and verification of a fully functional 4-bit MIPS-lit RISC processor using Skywater 130nm PDK in Cadence Virtuoso. Through a disciplined hierarchical methodology spanning transistor-level cells, functional blocks, and system-level integration, the team demonstrated the complete VLSI design flow, from architectural specification to GDSII-ready layout. Each major component, including the control unit, ALU, register file, and interconnect logic, was individually designed, laid out, and verified using Pegasus DRC, LVS and ERC, ensuring both logical correctness and physical design rule compliance.

The processor supports the full graduate instruction set of eight commands (READ, WRITE, ADD, SUB, AND, OR, SR, SL) operating over a 4-bit datapath with three registers. The final floorplan integrates all blocks into a compact, routable layout with aligned power grids and minimized interconnect delay, reflecting industry-relevant physical design practices.

Lessons Learned:

- Hierarchical design is essential for managing complexity and enabling parallel development.
- Early and incremental verification (DRC, LVS, ERC at each level) prevents integration failures and reduces debug time.
- Tool proficiency in Cadence Virtuoso and Pegasus is not merely operational but foundational to achieving sign-off quality in physical design.
- Practical experience in collaborative IC design, verification-driven development, and physical implementation, skills directly transferable to careers in semiconductor engineering and digital systems design.

References

[1] P. V. S. R. Bharadwaja, K. R. Teja, M. N. Babu, and K. Neelima, “Advanced low power RISC processor design using MIPS instruction set,” *2015 2nd International Conference on Electronics and Communication Systems (ICECS)*, pp. 1252–1258, Feb. 2015, doi: https://doi.org/10.1109/ecs.2015.7124785.

[2] P. Gautham, R. Parthasarathy and K. Balasubramanian, "Low-power pipelined MIPS processor design," *Proceedings of the 2009 12th International Symposium on Integrated Circuits*, Singapore, 2009, pp. 462-465.

[3] H. Mehta, “Design of MIPS Processor,” MS Thesis, California State University Northridge, 2012. Accessed: Dec. 03, 2025. [Online]. Available: https://scholarworks.calstate.edu/downloads/tb09j846s

[4] Y. Zhou and H. Guo, “Application Specific Low Power ALU Design,” in *2008 IEEE/IFIP International Conference on Embedded and Ubiquitous Computing*, IEEE, Dec. 2008, pp. 214–220. doi: https://doi.org/10.1109/euc.2008.81.

[5] N. H. E. Weste and David Money Harris, *Integrated circuit design*. Boston: Pearson / Addison Wesley, 2011.

[6] Praveen, G., Manasa, M., Chandana, S., & Jyothsna, T. (2023). *Design and analysis of a* 4-bit low-power universal barrel shifter using 2×1 MUX in 16 nm FinFET technology. International Journal for Research Trends and Innovation, 8(6), 734–739. https://www.ijrti.org/papers/IJRTI2306112.pdf

Appendix A: Layout and Schematic Figures

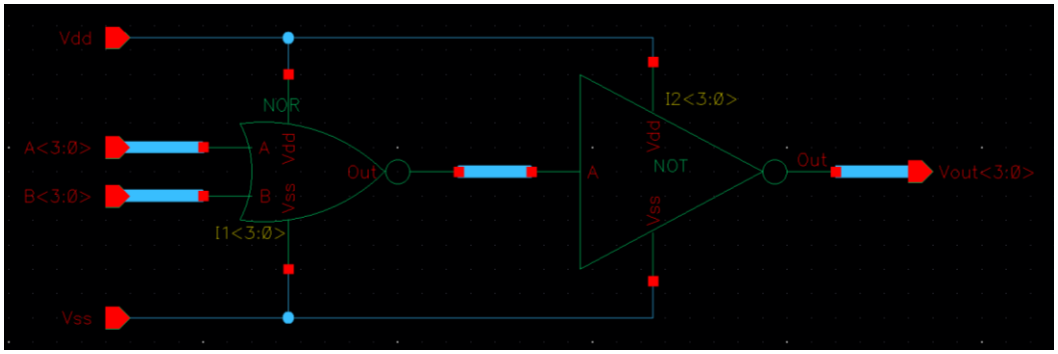

Figure 11 4Bit OR

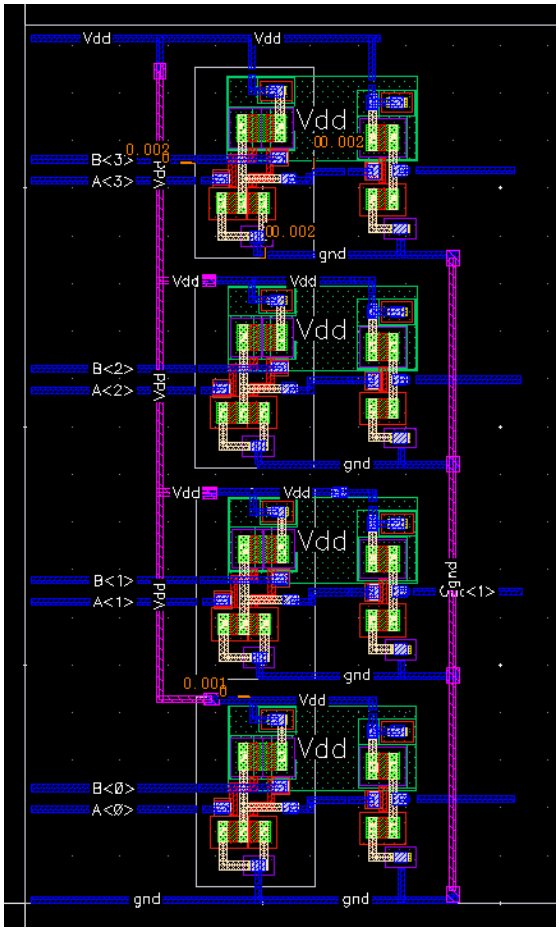

Figure 12 4Bit OR Layout

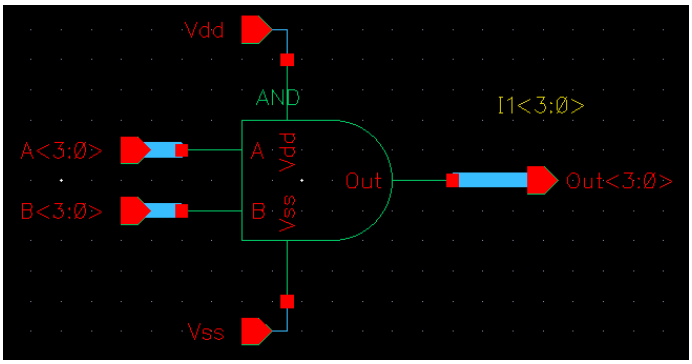

Figure 13 4Bit AND Schematic

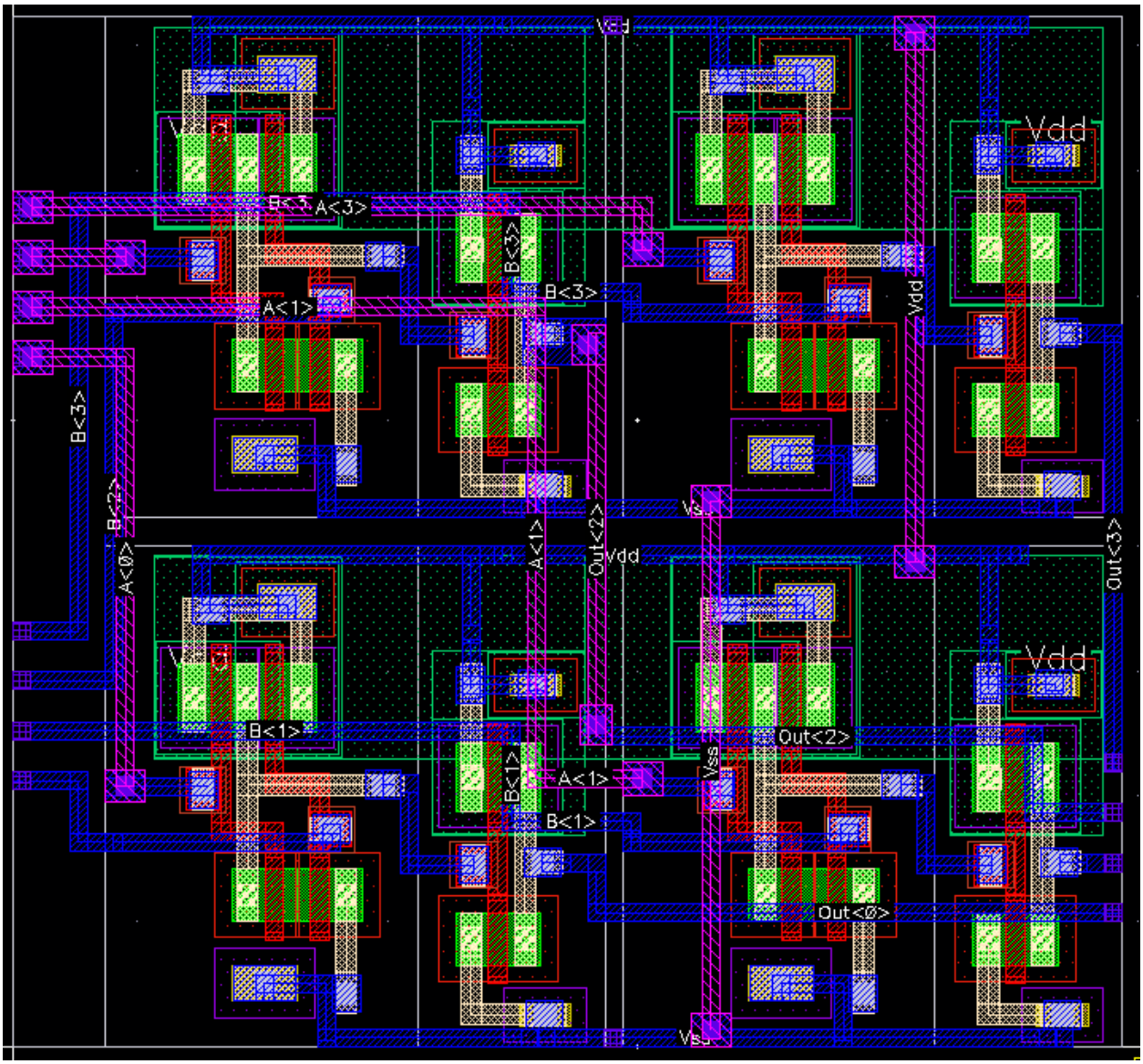

Figure 14 4Bit AND Layout

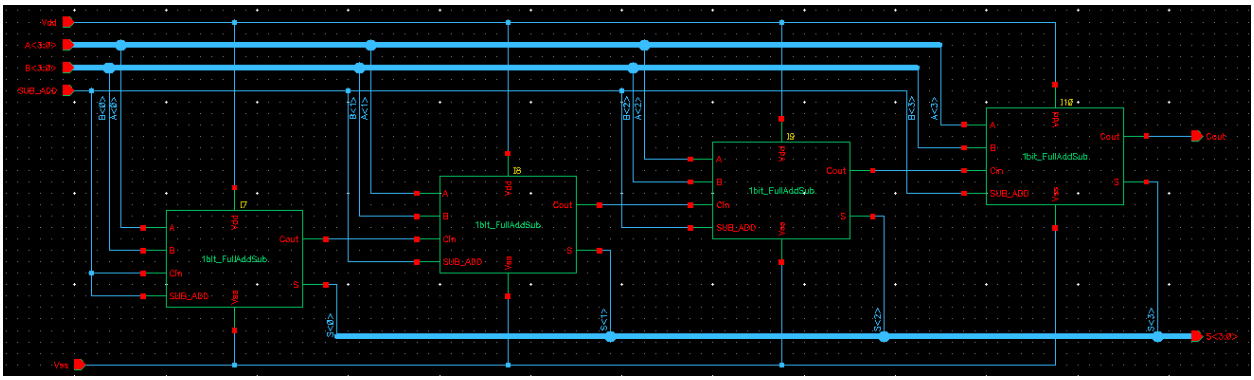

Figure 15 4-bit full adder with subtraction capability

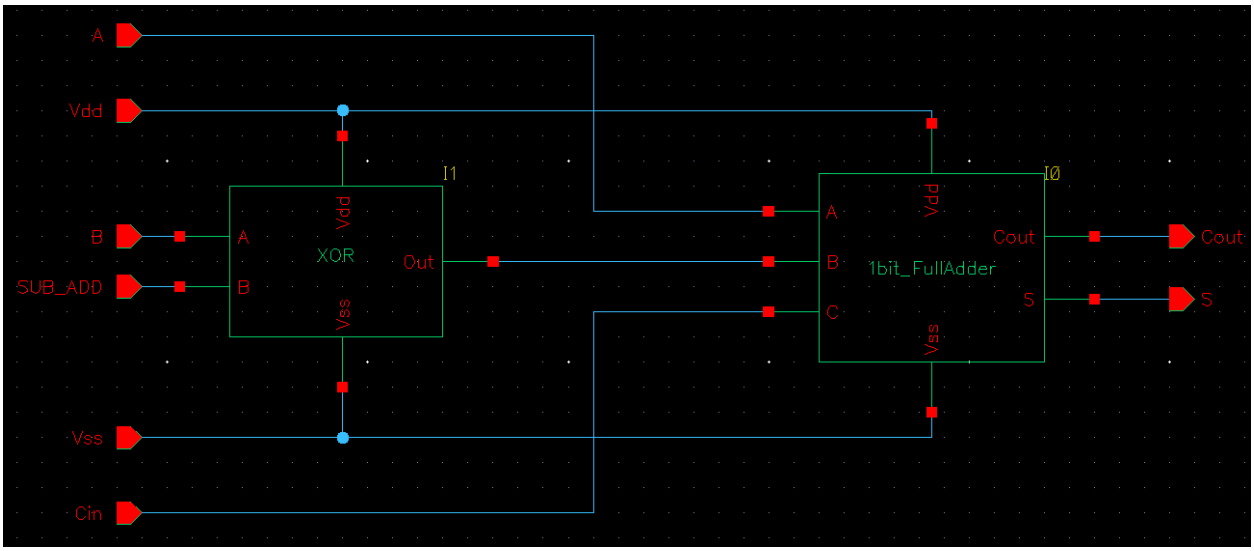

Figure 16 1-bit full adder with integrated XOR to facilitate subtraction

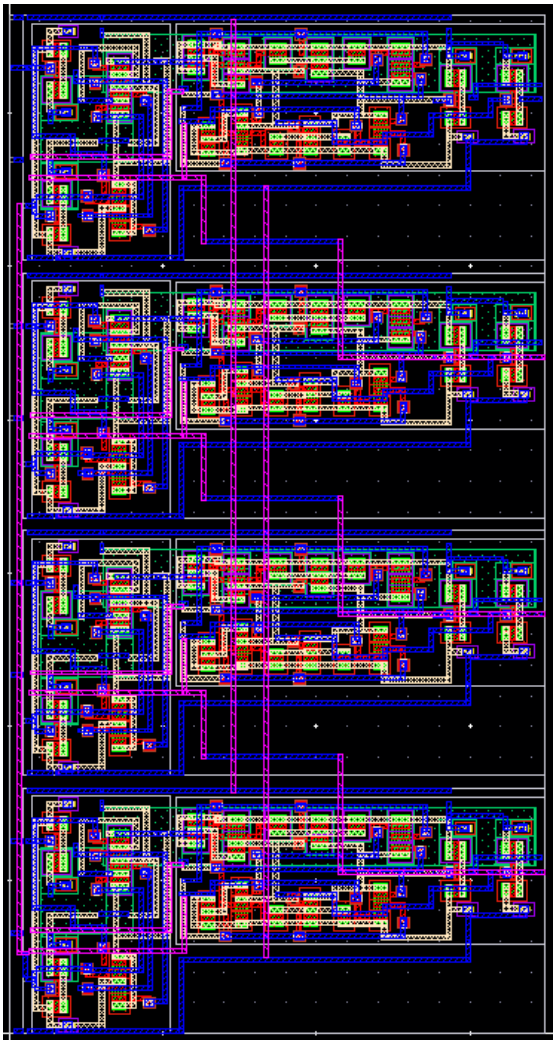

Figure 17 4-bit full adder layout

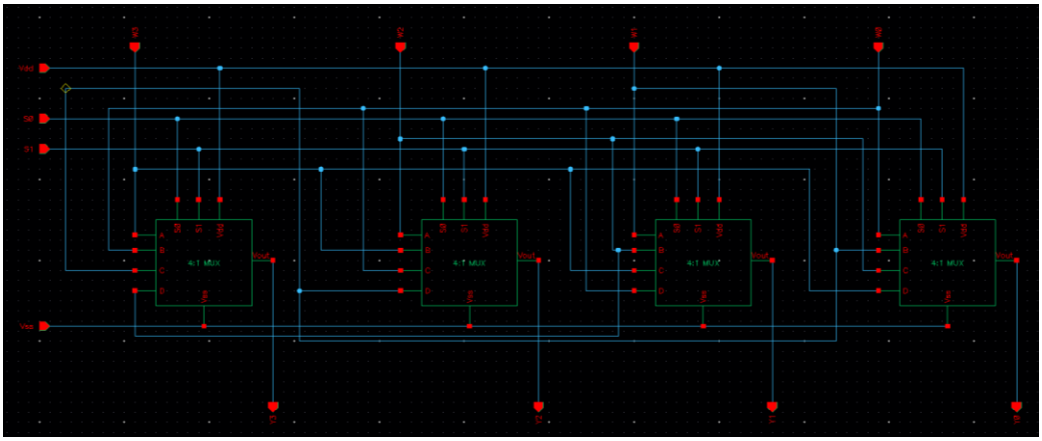

Figure 18 Barrel Shifter Schematic

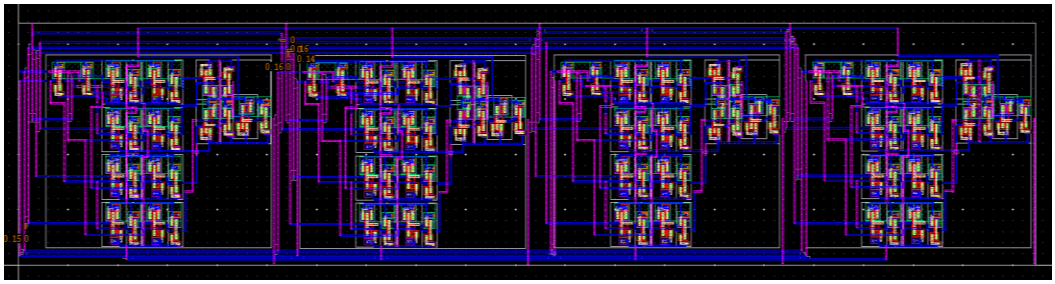

Figure 19 Barrel Shifter Layout

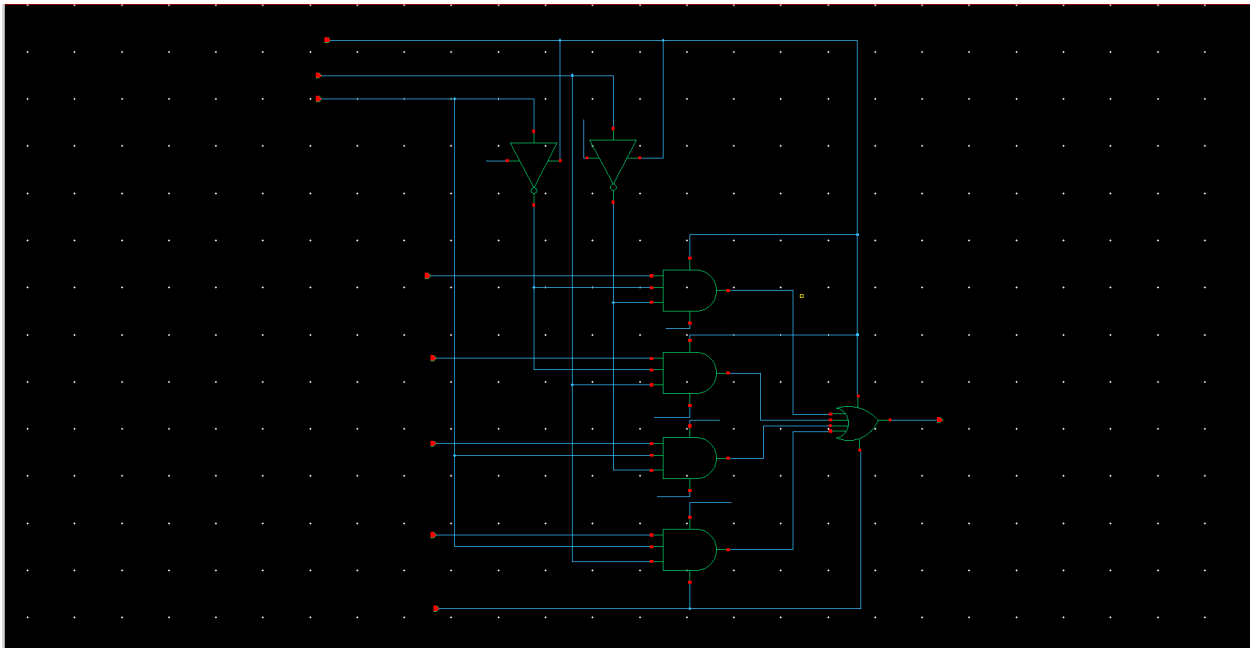

Figure 20 4:1 MUX schematic

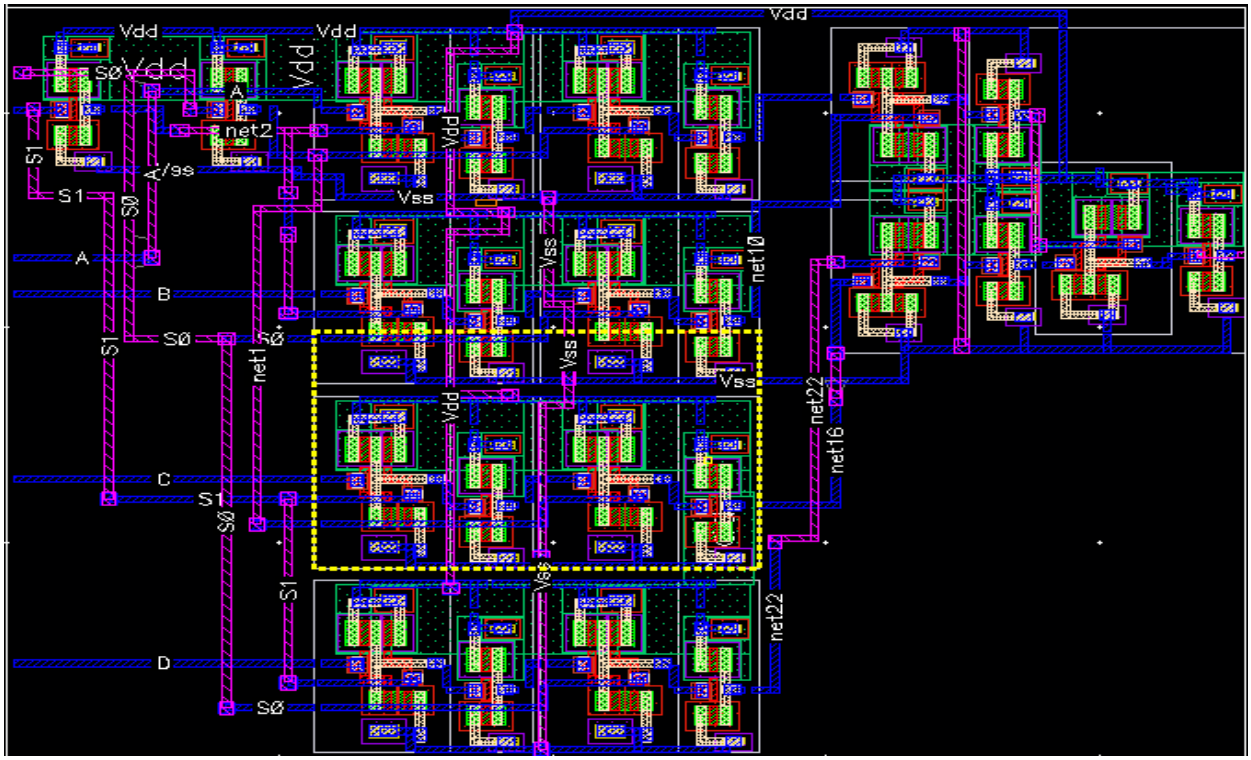

Figure 21 4:1 MUX Layout

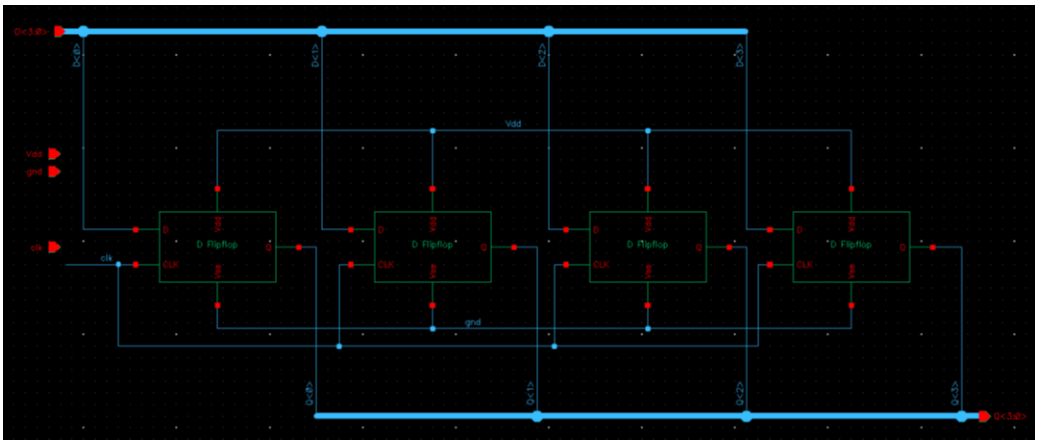

Figure 22 Register (4-bit) schematic

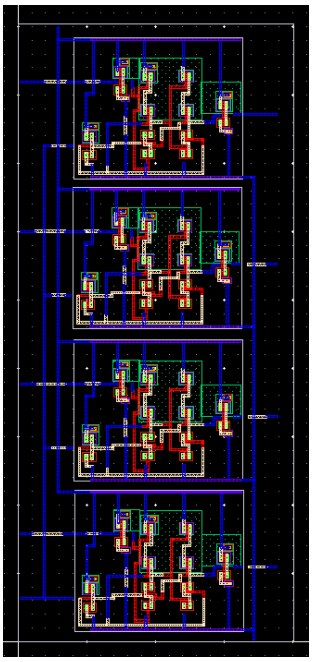

Figure 23 Register layout

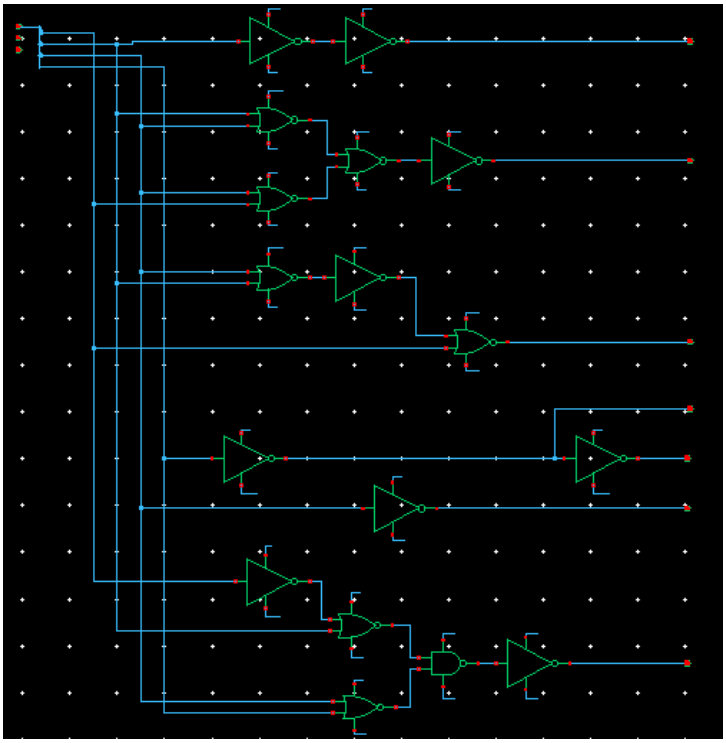

Figure 24 Control logic schematic schematic

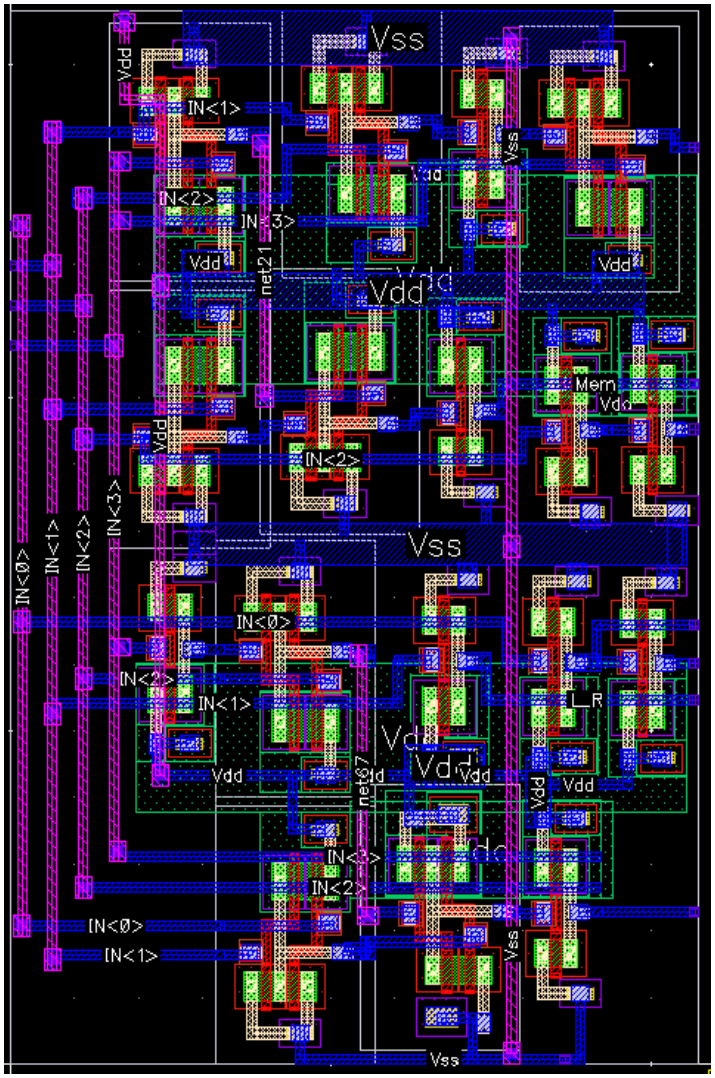

Figure 25 Control Logic layout

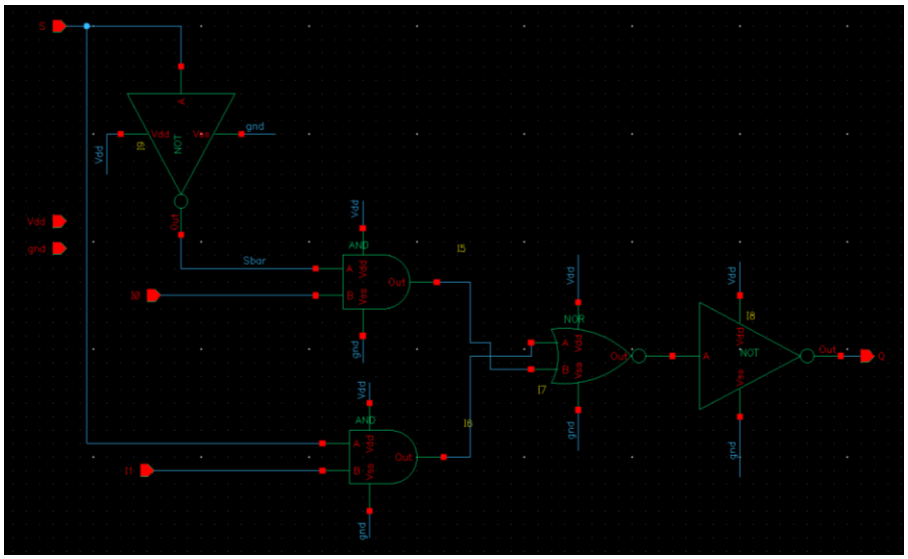

Figure 26 2:1 MUX schematic

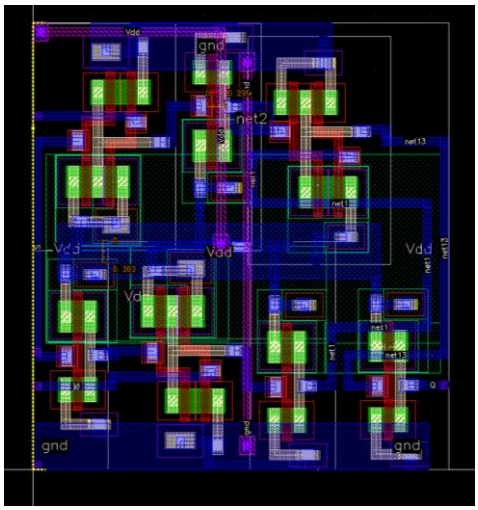

Figure 27 2:1 MUX Layout

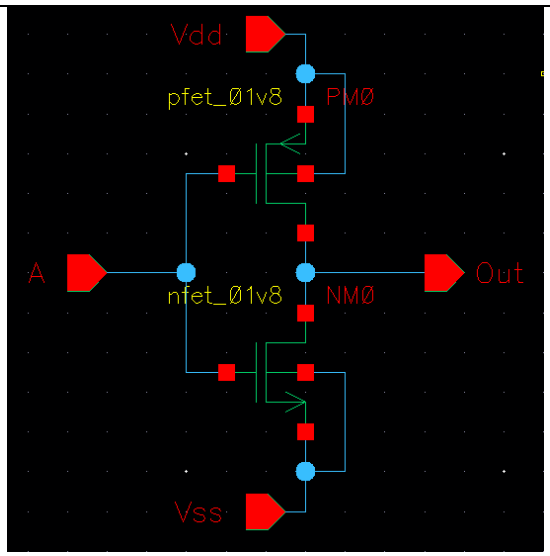

Figure 28 NOT Schematic

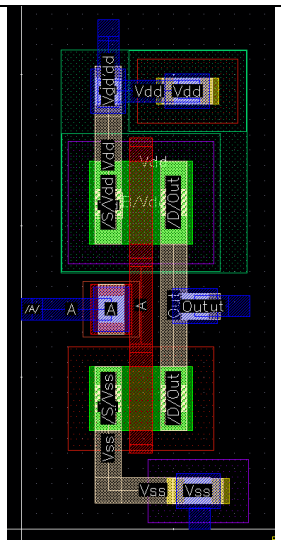

Figure 29 NOT Layout

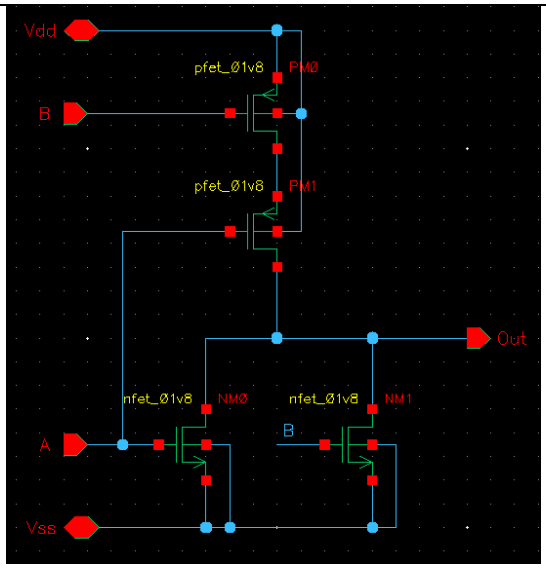

Figure 30 NOR Schematic

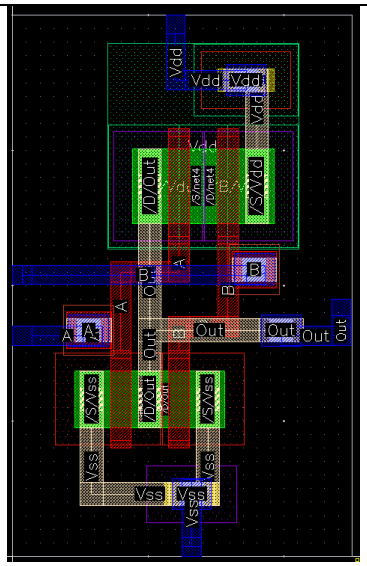

Figure 31 NOR Layout

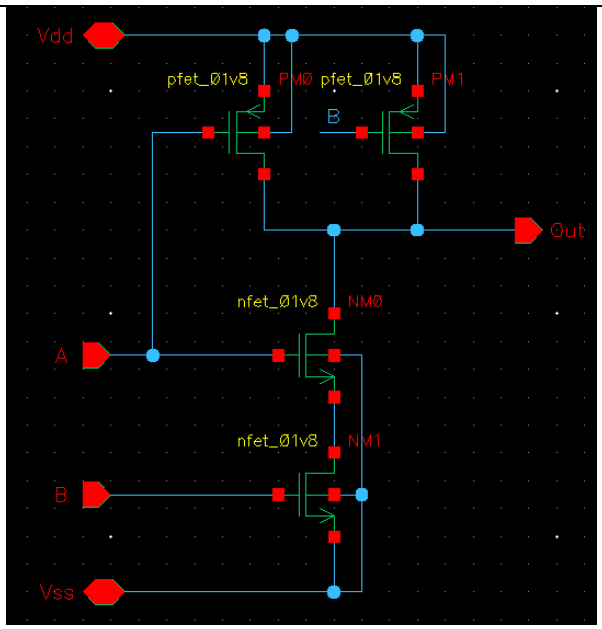

Figure 32 NAND Schematic

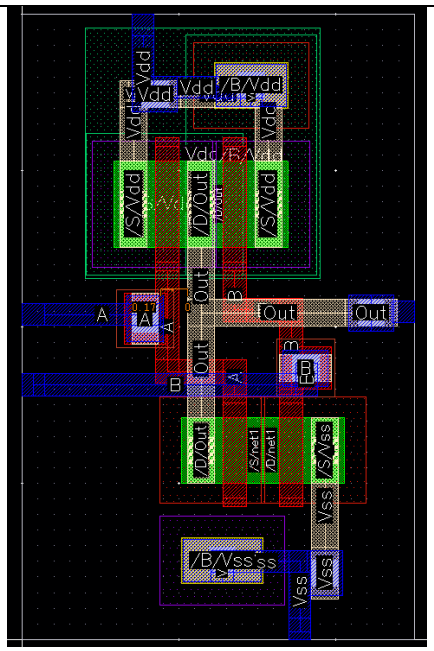

Figure 33 NAND Layout

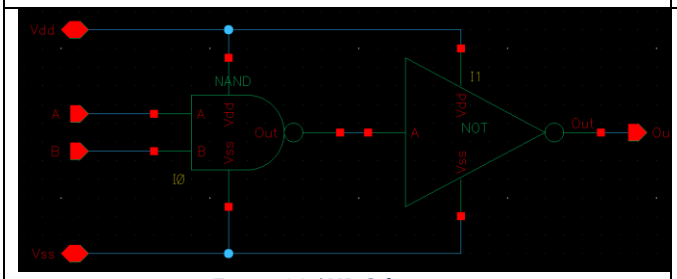

Figure 34 AND Schematic

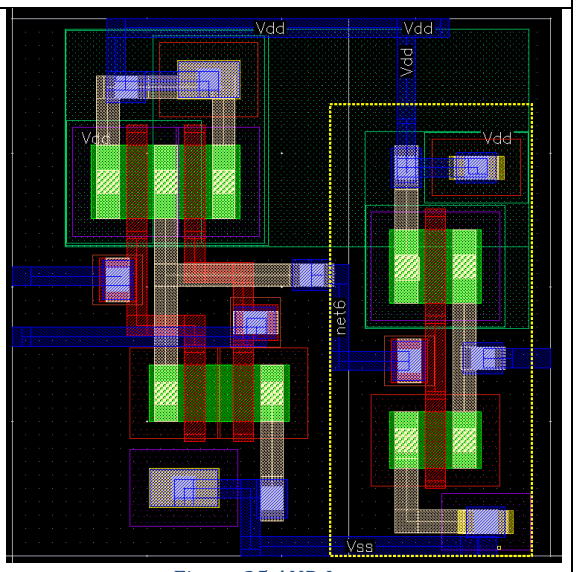

Figure 35 AND Layout

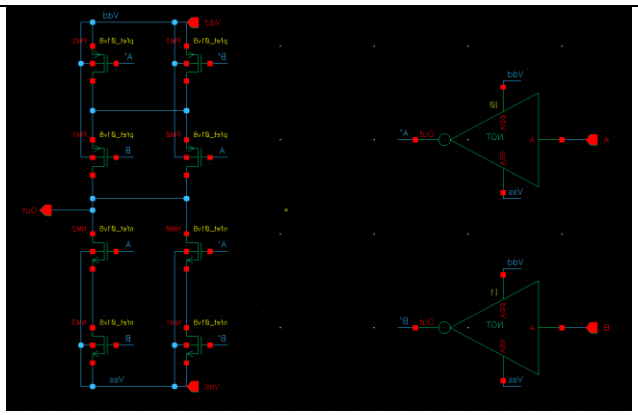

Figure 36 XOR Schematic

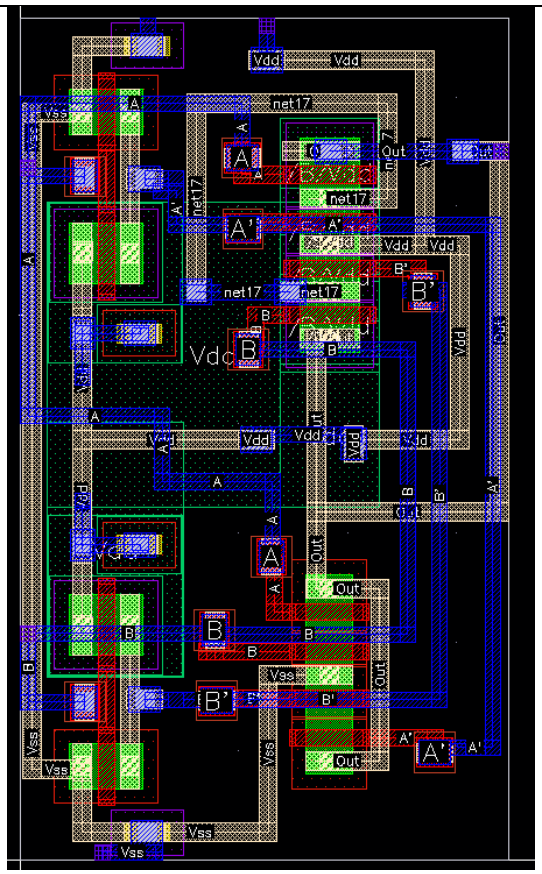

Figure 37 XOR Layout

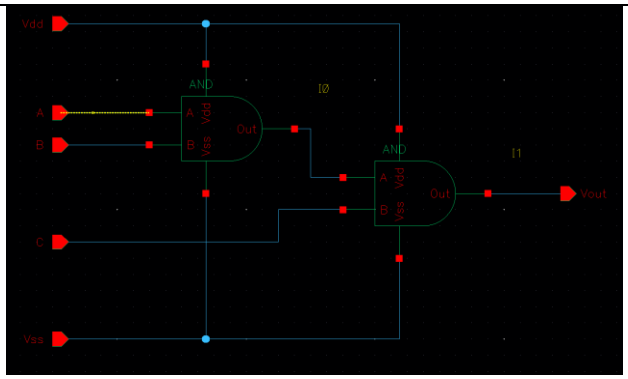

Figure 38 AND3 Schematic

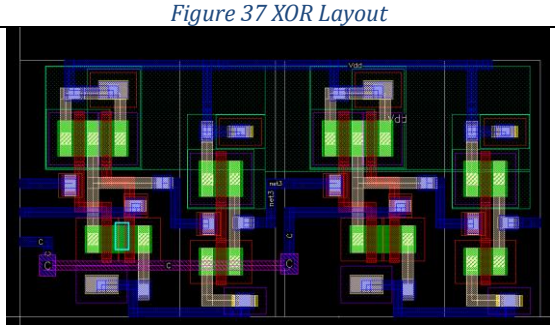

Figure 39 AND3 Layout

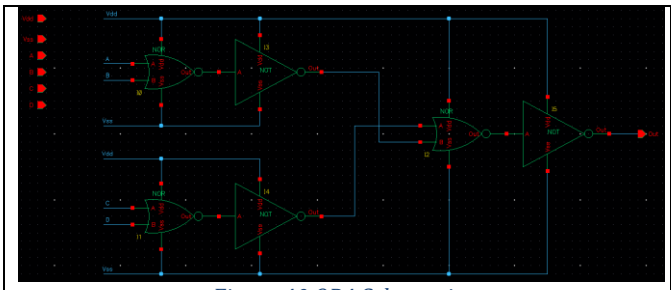

Figure 40 OR4 Schematic

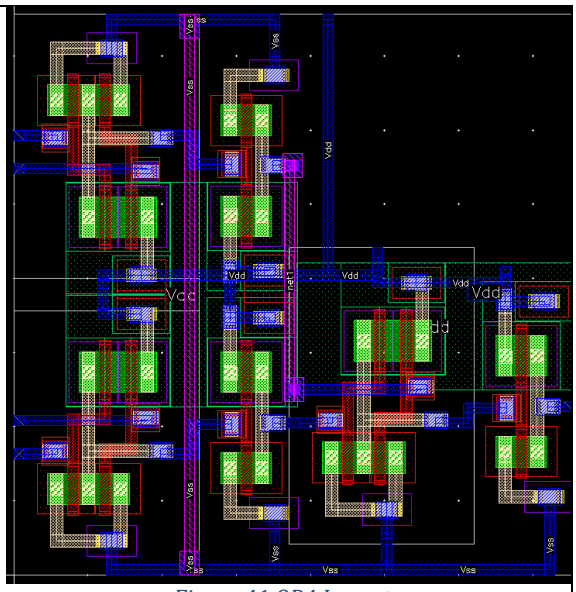

Figure 41 OR4 Layout

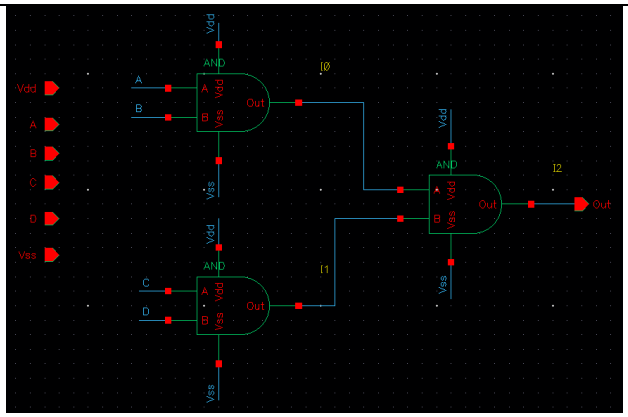

Figure 42 AND4 Schematic

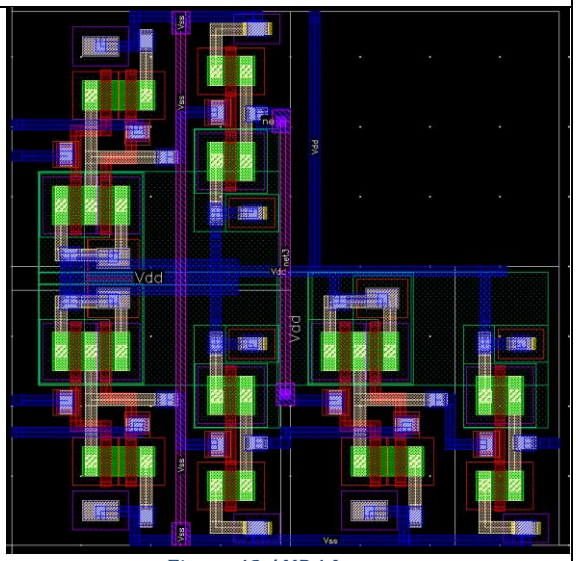

Figure 43 AND4 Layout